Laporan Solusi AI Agent untuk Industri Farmasi

Analisis Existing Solutions & Rekomendasi Implementasi

Executive Summary

Industri farmasi menghadapi empat tantangan operasional utama yang dapat diselesaikan dengan AI Agent dan teknologi modern:

1. **Otomatisasi Dokumen & Kepatuhan Regulasi** - Mempercepat dokumen regulatory dari berhari-hari menjadi menit
2. **Pencarian Data Knowledge Base** - Membuat informasi dari ribuan dokumen dapat diakses melalui conversational AI
3. **Visibility Supply Chain** - Tracking distribusi obat dengan visualisasi geospasial real-time
4. **Manajemen Data Internal** - Aplikasi modern untuk tugas administratif departemen kecil

Laporan ini menganalisis **15+ existing solutions** di GitHub dan platform komersial, dengan fokus pada implementasi menggunakan **AI Agent dan LLM-RAG architecture**.

---

1. Otomatisasi Dokumen & Kepatuhan Regulasi

1.1 Problem Statement

Big Pharma menghasilkan ribuan dokumen per hari mencakup:

- Laporan uji klinis (Clinical Trial Reports)
- Standard Operating Procedures (SOP)
- Pengajuan izin edar (510k, IND/CTA, NDA submissions)

- Compliance documentation

Waktu yang terbuang: 5-14 hari untuk drafting dokumen kompleks secara manual.

1.2 Existing Solutions yang Tersedia

A. Commercial Solutions [cite:1]

Everest Platform (AI-Powered Document Generation)

- **Fitur:** Generasi otomatis 510(k) filing, IND/CTA briefing packages
- **Performa:** Near-final drafts dalam menit, 40% lebih cepat
- **Keunggulan:** Pre-built templates untuk regulatory documents, compliance checks built-in
- **Keterbatasan:** Closed-source, commercial pricing

Arahi AI (Document Review & Automation)

- **Fitur:** Document review automation, regulatory compliance checking, version comparison
- **Performa:** Mengurangi manual effort hingga 85%
- **Keunggulan:** 1,500+ integrasi, workflow customization, enterprise security (SOC 2 compliant)
- **Keterbatasan:** No-code platform, less customizable untuk needs spesifik

B. Open-Source Solutions [cite:2]

Nama: AIAgents4Pharma (VirtualPatientEngine)

- **GitHub:** VirtualPatientEngine/AIAgents4Pharma
- **Tech Stack:** AI Agents (LangChain), LLM integration
- **Status:** Active development (2024+)
- **Fokus:** Drug research interaction, AI-driven tools untuk researchers
- **Kekuatan:** Purpose-built untuk pharma, agentic framework
- **Limitation:** Fokus research, belum fully untuk regulatory documents

Nama: Posit Platform (R/Python Clinical Analytics)

- **Framework:** Open-source R/Python dengan governance

- **Adoption:** Used by Roche, GSK, Novo Nordisk
- **Keunggulan:** FDA-accepted, reproducible workflows, GxP-compliant
- **Use Case:** SDTM/ADaM dataset generation, submission-ready analytics [cite:3]
- **Keterbatasan:** Lebih untuk data processing, bukan document generation pure

1.3 Rekomendasi Implementasi untuk Pharma

Best Approach: Hybrid Open-Source + Custom Agent

```
Stack yang Recommended:
├── Backend: Python + FastAPI
├── LLM Integration: Google Gemini API atau Claude
├── Document Templates: Jinja2 template engine
├── Data Validation: Pydantic + custom validators
├── AI Agent: LangChain with Tool-Use capabilities
├── Database: PostgreSQL + vector DB (ChromaDB) untuk template embedding
└── Frontend: React + Next.js untuk form submission
```

Architecture Workflow:

```
User Input (Form/Upload)
↓
[Data Validation & Extraction Agent]
├─ Parse form fields atau extract dari uploaded file
├─ Validate terhadap regulatory requirements
└─ Flag missing/invalid data
↓
[Template Mapping Agent]
├─ Search base knowledge untuk document type yang tepat
├─ Retrieve relevant template sections
└─ Identify required & optional sections
↓
[Content Generation Agent]
├─ Use LLM untuk generate regulatory-compliant text
├─ Maintain formatting standards (ISO 11135, 21 CFR Part 11)
└─ Cross-reference dengan existing approvals
↓
```

[Quality Gate Agent]
├─ Compliance checker: regulatory requirement coverage
├─ Consistency checker: data validation across sections
├─ Formatting checker: template adherence
└─ Plagiarism/Risk detector
↓
Output: PDF/DOCX near-final draft

Implementation Timeline:

- Phase 1 (2 minggu): Set up infrastructure + pilot untuk 1 document type (SOP)
- Phase 2 (4 minggu): Add 510(k) template + regulatory validators
- Phase 3 (3 minggu): Integration + testing dengan actual pharma workflows

---

2. Pencarian Data Knowledge Base Raksasa dengan RAG

2.1 Problem Statement

Tim R&D kesulitan mencari informasi dari:

- Ribuan penelitian papers (PubMed, internal publications)
- Catatan uji klinis (clinical trial notes, patient data summaries)
- Guidelines tebal (ratusan halaman regulatory guidelines)
- Internal documentation (past projects, approved submissions)

Pencarian keyword tradisional kurang efektif untuk konteks yang kompleks.

2.2 Existing Solutions yang Tersedia

A. Production-Ready RAG Systems untuk Medical [cite:4][cite:5]

1. Medical-Graph-RAG (Chrisolande)

- **GitHub:** Chrisolande/Medical-Graph-RAG
- **Tech:** Python + LangChain + FAISS + Knowledge Graph
- **Features:**
  - Knowledge graph construction dari documents

    - Vector search dengan reranking (Jina AI, FlashRank)
    - Semantic caching (LLM response caching)
    - Streamlit interactive UI
    - Support PMC JSON, pre-chunked formats
- **Performa:** Real-time graph traversal visualization, conversation history
- **Kelebihan:** Comprehensive, production-tested, modular design
- **Status:** Active, MIT licensed

2. Medical-RAG-LLM (THE-DEEPDAS)

- **GitHub:** THE-DEEPDAS/Medical-RAG-LLM
- **Features:** Comprehensive medical assistant bot, locally deployed LLM
- **Use Case:** Real-time analysis, privacy-preserving (local deployment)
- **Kelebihan:** On-premise option, no data leaving infrastructure

3. Drug-RAG (Maastricht University)

- **GitHub:** MaastrichtU-IDS/Drug-RAG
- **Features:** Multiple LLM testing, RAG technique comparison, expert evaluation
- **Status:** Recently updated (Feb 2025)
- **Kelebihan:** Research-backed, LLM optimization framework

4. PharmaQuery (CodeWithCharan)

- **GitHub:** CodeWithCharan/PharmaQuery
- **Tech Stack:**
    - LLM: Google Gemini API (gemini-1.5-pro)
    - Embeddings: Google Gemini (embedding-001)
    - Database: ChromaDB
    - Document processing: PyPDFLoader
    - Text splitting: SentenceTransformersTokenTextSplitter
- **Features:** Pharmaceutical Insight Retrieval System
- **Status:** Recently updated (Jan 2025), Python 3.10+
- **Kelebihan:** Modern tech stack, Gemini integration, production-ready

B. MedRAG Toolkit [cite:6]

MedRAG (Comprehensive Medical QA Toolkit)

- **GitHub:** teddy-xionggz/medrag
- **Purpose:** Systematic toolkit untuk RAG on medical QA
- **Implementation:** Various RAG systems untuk medical question answering
- **Research-backed:** Published toolkit dengan proven methodologies

2.3 Rekomendasi Implementasi untuk Pharma

Best Approach: Enterprise RAG dengan AI Agent Orchestration

```
Architecture Stack:
├── Document Ingestion
│  ├─ PDF Parser (PyPDF2, pdfplumber)
│  ├─ Document Chunker (recursive splitting)
│  └─ Metadata extractor (author, date, classification)
│
├── Vector Database
│  ├─ ChromaDB atau Pinecone (vector storage)
│  ├─ Embedding Model: Google Gemini embedding-001
│  └─ Index: ~10K documents per collection
│
├── Knowledge Graph Layer
│  ├─ Entity extraction (drugs, conditions, regulations)
│  ├─ Relationship mapping (drug-indication, regulation-requirement)
│  └─ Graph database: Neo4j optional for complex queries
│
├── AI Agent Orchestration
│  ├─ Query Router: Route to appropriate knowledge source
│  ├─ Retriever Agent: Find relevant documents
│  ├─ Extractor Agent: Pull specific facts with citations
│  ├─ Comparator Agent: Highlight inconsistencies
│  └─ Validator Agent: Ensure accuracy & compliance
│
├── LLM Integration
│  ├─ Primary: Google Gemini 1.5 Pro (best for pharma context)
│  ├─ Fallback: Claude 3.5 Sonnet
```

```
│  └─ Local option: Llama 2 70B for sensitive data
│
├── Interface
│  ├─ Chat UI (React + WebSocket)
│  ├─ Document viewer dengan highlight
│  ├─ Citation tracker
│  └─ Conversation history
│
└── Backend
├─ FastAPI server
├─ LangChain orchestration
├─ PostgreSQL untuk user data
└── Redis untuk caching
```

Workflow Conversational AI:

```
User Query: "What are the known drug interactions between Aspirin
and Warfarin?"
↓
[Query Router Agent]
├─ Classify: Drug interaction query
└─ Route to: Medical knowledge base
↓
[Retriever Agent]
├─ Vector search: Find Aspirin-Warfarin interaction docs
├─ Graph search: Find related contraindications
└─ Retrieve: Top 5 most relevant sections with score
↓
[Reranker]
├─ Re-score based on relevance
└─ Final ranking: Top 3 sections
↓
[LLM Generation with Agent]
├─ Prompt: "Summarize known interactions with citations"
├─ Context: Top 3 retrieved sections
├─ Tools available:
│  ├─ reference_document(doc_id)
│  ├─ check_recent_studies(query)
│  └─ compare_guidelines(guideline_a, guideline_b)
└─ Output: Structured answer with references
```

↓
[Validator Agent]
├─ Fact-check: All claims against source documents
├─ Confidence scoring: High/Medium/Low
└─ Flag: Any contradictions with other sources
↓
Response to User:
"Based on pharmaceutical literature [cite:7][cite:8]:

- Warfarin + Aspirin: SIGNIFICANT interaction (increased bleeding risk)
- Clinical recommendation: Require close INR monitoring or use alternative
- Source: FDA Orange Book, UpToDate, PubMed Central
- Last updated: [timestamp]"

Implementation Phases:

- Phase 1 (3 minggu): Setup vector DB + ingest first 1000 documents
- Phase 2 (2 minggu): Implement query router + basic RAG
- Phase 3 (3 minggu): Add knowledge graph + validation agents
- Phase 4 (2 minggu): Deploy chat UI + testing

Cost Estimate (Monthly):

- Gemini API: ~$200-500 (10K queries)
- Vector DB (Pinecone): ~$100 (100M tokens)
- Infrastructure (AWS): ~$300-500
- **Total:** ~$600-1000/month untuk enterprise setup

---

3. Visibility Rantai Pasok (Supply Chain) & Distribusi

3.1 Problem Statement

Challenges dalam supply chain pharma:

- Tracking vaksin/obat sensitif-suhu dari pabrik → distribusi → rumah sakit

- Visibilitas bottleneck pengiriman real-time
- Pemetaan kebutuhan per daerah (demand forecasting)
- Compliance dengan cold chain requirements

3.2 Existing Solutions yang Tersedia

A. Geospatial Analytics + BI Framework [cite:9][cite:10]

Integration Approach: Geospatial BI + Supply Chain Optimization

Key Technologies Available:

1. **GIS Software:** ArcGIS, QGIS, Folium (Python)
2. **Analytics Dashboards:** Power BI, Tableau, Metabase
3. **Database:** PostGIS (PostgreSQL with spatial data)
4. **Real-time tracking:** IoT sensors + GPS integration

Academic Framework (2016-2026):

- Location-based demand forecasting
- Route optimization dengan travel-time analysis
- Territory management visualization
- Inventory movement pattern analysis

B. Open-Source Supply Chain Projects [cite:11]

pharmaceutical-supply-chain-agentic-ai (GitHub)

- **GitHub:** MahdiNavaei/pharmaceutical-supply-chain-agentic-ai
- **Architecture:** Agentic AI system untuk pharma supply chain
- **Focus:** Management + optimization dengan AI agents
- **Status:** Comprehensive project, actively maintained
- **Features likely include:**
  - Inventory tracking
  - Route optimization
  - Demand forecasting
  - Real-time alerts

MapWeave Supply Chain Geospatial Demo [cite:12]

- **Purpose:** End-to-end supply chain geospatial network analysis
- **Features:**
  - Full supply chain visibility

    - Network analysis
    - Real-time dashboards
    - Geographic routing

3.3 Rekomendasi Implementasi untuk Pharma

Best Approach: AI Agent + Geospatial BI + Real-time Tracking

```
Complete Architecture:
├── Data Collection Layer
│  ├─ IoT Sensors (temperature, humidity, GPS)
│  ├─ API integrations (warehouse systems, transport providers)
│  ├─ Manual data entry (fallback)
│  └─ Aggregator: Kafka streams untuk real-time
│
├── Data Storage
│  ├─ Time-series DB: InfluxDB (sensor data)
│  ├─ Spatial DB: PostGIS (location data)
│  ├─ Document DB: MongoDB (metadata)
│  └─ Cache: Redis (real-time queries)
│
├── AI Agent System
│  ├─ Demand Forecasting Agent
│  │  ├─ Input: Historical sales + population data
│  │  ├─ Tool: Prophet/ARIMA forecasting
│  │  └─ Output: Demand prediction per region
│  │
│  ├─ Route Optimization Agent
│  │  ├─ Input: Distribution points, vehicles, constraints
│  │  ├─ Tool: Traveling Salesman Problem (TSP) solver
│  │  └─ Output: Optimal routes + ETA
│  │
│  ├─ Inventory Management Agent
│  │  ├─ Input: Current stock, demand forecast, lead times
│  │  ├─ Tool: Safety stock calculation, reorder point logic
│  │  └─ Output: Reorder alerts + recommendations
│  │
│  ├─ Cold Chain Monitoring Agent
│  │  ├─ Input: Temperature sensors, compliance rules
│  │  ├─ Tool: Deviation alerts, escalation rules
```

```
│ │ └─ Output: Temperature breach notifications
│ │
│ └─ Anomaly Detection Agent
│ ├─ Input: Real-time tracking data
│ ├─ Tool: Statistical anomaly detection (Isolation Forest)
│ └─ Output: Alert unusual patterns (delays, theft risk)
│
├── Visualization & Analytics
│ ├─ Interactive Map (Leaflet/Mapbox)
│ │ ├─ Real-time vehicle tracking
│ │ ├─ Heatmap: Demand vs supply by region
│ │ ├─ Density map: Hospital/clinic distribution
│ │ └─ Route visualization
│ │
│ ├─ Dashboard (React + D3.js)
│ │ ├─ KPIs: On-time delivery %, stock-out incidents
│ │ ├─ Bottleneck analysis: Where delays happen
│ │ ├─ Cost analysis: Transportation + storage cost
│ │ └─ Compliance: Cold chain violations
│ │
│ └─ Predictive Alerts
│ ├─ "Stock-out risk in Region X in 3 days"
│ ├─ "Recommended shipment of Y units to Z"
│ └─ "Temperature breach detected, ETA alert"
│
└── Backend Infrastructure
├─ FastAPI service (agents)
├─ Apache Airflow (workflow orchestration)
├─ LangChain (agent orchestration)
└─ Google Cloud Run atau AWS Lambda (scalable compute)
```

Example Use Case Workflow:

Scenario: Jakarta warehouse inventory running low for antimalarial drugs

```
[Real-time Trigger]
Stock level < safety stock threshold
↓
[Inventory Agent]
```

├─ Check current inventory: 500 units
├─ Forecast demand (next 7 days): 300 units/day
├─ Calculate safety stock: 1500 units
├─ Decision: REORDER NEEDED
↓
[Demand Forecasting Agent]
├─ Analyze regional demand pattern
├─ Check population density: Jakarta (10M) vs Bandung (2.5M)
├─ Historical seasonality: Malaria peak = Oct-Nov
├─ Recommendation: 3000 units untuk Jakarta, 1000 units untuk Bandung
↓
[Route Optimization Agent]
├─ Identify optimal distribution centers
├─ Calculate travel time: Factory → DC1 (4h) vs DC2 (3.5h)
├─ Consider temperature constraints (malaria drugs require 2-8°C)
├─ Recommend: Route via DC2 with cold truck
↓
[AI Agent Executive Summary]
Output:
"ALERT: Jakarta stock-out risk in 1.5 days
Recommendation:

- Ship 3000 units via cold truck
- Route: Factory → Distribution Center B → Jakarta warehouse
- ETA: 12 hours
- Cost: $500, Estimated arrival: 2:30 PM today
- Cold chain compliance: FULL (2-8°C maintained)
- Expected delivery: 2:30 PM today
  Approve? [YES/NO]"

Implementation Phases:

- Phase 1 (4 minggu): Setup databases + data collection infrastructure
- Phase 2 (3 minggu): Implement demand forecasting + inventory agents
- Phase 3 (3 minggu): Add route optimization + real-time tracking
- Phase 4 (2 minggu): Build geospatial dashboard + testing

---

4. Manajemen Data Internal & Aplikasi Skala Kecil

4.1 Problem Statement

Departemen kecil/spesifik masih menggunakan:

- Excel/spreadsheet legacy lambat
- Sistem old-fashioned untuk tracking task
- Kesulitan request software baru ke IT pusat (birokratis)

Use cases:

- Inventory lab management
- Task tracking harian
- Daily operations management

4.2 Existing Solutions

A. AI Task Manager (GitHub) [cite:13]

AI-Task-Manager (itallonardi)

- **GitHub:** itallonardi/AI-task-manager
- **Tech:** LangChain agent untuk task management
- **Features:** LLM-powered task orchestration
- **Use Case:** Example untuk agentic task workflows

B. Lightweight Internal Tools Stack

Recommended modern stack untuk departemen kecil:

- **Frontend:** React + Next.js (modern, fast, easy to deploy)
- **Backend:** FastAPI + PostgreSQL (simple, powerful, fast)
- **Auth:** NextAuth.js atau Firebase Auth (no code authentication)
- **Deployment:** Vercel + Railway atau DigitalOcean App Platform ($10-30/month)

4.3 Rekomendasi Implementasi

Best Approach: Low-Code Internal Tool dengan AI Assistants

```
Tech Stack:
├── Frontend (React + Next.js)
│   ├─ Dashboard untuk inventory/task overview
│   ├─ Form builder untuk data entry
│   ├─ Real-time notifications
│   └─ Mobile responsive
│
├── Backend (FastAPI)
│   ├─ RESTful APIs untuk CRUD operations
│   ├─ Job queue (Celery) untuk background tasks
│   └─ WebSocket untuk real-time updates
│
├── Database (PostgreSQL)
│   ├─ Inventory table (item, quantity, location)
│   ├─ Task table (title, assignee, status, deadline)
│   ├─ Users table (role-based access)
│   └─ Audit log (for compliance)
│
├── AI Integration (Optional but Powerful)
│   ├─ Inventory Reminder Agent
│   │   └─ "Stock level of reagent X below 10%, reorder?"
│   │
│   ├─ Task Prioritization Agent
│   │   └─ "Based on deadlines, prioritize: Task A > Task B > Task C"
│   │
│   ├─ Anomaly Detection Agent
│   │   └─ "Usage of reagent Y is 3x higher than normal, why?"
│   │
│   └─ Report Generation Agent
│   └─ Auto-generate monthly inventory report
│
└── Deployment (Minimal Cost)
    ├─ Frontend: Vercel (free tier atau $20/month)
    ├─ Backend: Railway (pay-as-you-go, ~$10-20/month)
    ├─ Database: Railway PostgreSQL (~$15/month)
    └─ Total: ~$45/month all-in
```

Features untuk Internal Use:

Dashboard Example (Inventory Management):

```
┌──────────────────────────────────────────┐
│ Lab Inventory System │
├──────────────────────────────────────────┤
│ [Search] [Add Item] [Export] │
├──────────────────────────────────────────┤
│ Item | Qty | Min | Status│
├──────────────────────────────────────────┤
│ Reagent A | 50 | 20 | ✓ OK │
│ Reagent B | 5 | 20 | ⚠ LOW │
│ Equipment X | 2 | 1 | ✓ OK │
│ Equipment Y | 0 | 1 | ✗ OUT │
└──────────────────────────────────────────┘
```

Smart Actions:

- Click "LOW": Auto-draft purchase request
- AI suggests: "Based on usage pattern, order 30 units"
- Notification: "Reagent B arriving today, 2 PM"

Ringkasan Rekomendasi & Action Items

Solusi	Status	Tech Stack	Cost (Monthly)	Timeline
Document Automation	Open-source available	Python + LLM + Jinja2	$100-300	6-8 minggu
Knowledge Base RAG	Production-ready	LangChain + ChromaDB + Gemini	$600-1000	8-10 minggu
Supply Chain Tracking	Partial (need assembly)	PostGIS + Geospatial + Agent	$500-1000	10-12 minggu
Internal Tools	Easy to build	FastAPI + React	$45-100	2-4 minggu

Key Takeaways

✓ **Semua 4 solusi layak dikembangkan** dengan existing open-source components
✓ **AI Agent framework siap**: LangChain + Gemini API sudah mature
✓ **GitHub resources available**: Multiple pharma-specific projects exist
✓ **Commercial alternatives exist** tapi lebih mahal & kurang flexible

Recommended Execution Order

1. **Quick Win (2-4 minggu):** Internal Tools (departemen kecil)
2. **High ROI (6-8 minggu):** Document Automation (save days per submission)
3. **Strategic Asset (8-10 minggu):** Knowledge Base RAG (competitive advantage)
4. **Operational Excellence (10-12 minggu):** Supply Chain Tracking (cost reduction)

Next Steps

1. Assess current infrastructure & team capabilities
2. Start dengan 1-2 pilot projects (Internal Tools + Document Automation)
3. Build internal AI engineer team atau hire consultants
4. Establish data governance + compliance framework
5. Scale to other departments

References

[1] Tely.ai (2025). AI Solutions for Streamlining Pharmaceutical Document Compliance. Retrieved from https://examples.tely.ai/pharma-biotech/10-ai-solutions-for-streamlining-pharmaceutical-document-compliance/

[2] GitHub Topics. AI Document Generation. Retrieved from https://github.com/topics/ai-document-generation

[3] Appsilon.com (2025). How Open Source Levels the Playing Field in Pharma. Retrieved from https://www.appsilon.com/post/how-global-pharma-leaders-use-open-source

[4] Chrisolande. Medical-Graph-RAG: Comprehensive RAG Pipeline. Retrieved from https://github.com/Chrisolande/Medical-Graph-RAG

[5] THE-DEEPDAS. Medical-RAG-LLM. Retrieved from https://github.com/THE-DEEPDAS/Medical-RAG-LLM

[6] Xionggz, T. (2024). MedRAG: Systematic Toolkit for RAG on Medical QA. Retrieved from https://github.com/teddy-xionggz/medrag

[7] Navaei, M. Pharmaceutical Supply Chain Agentic AI. Retrieved from https://github.com/MahdiNavaei/pharmaceutical-supply-chain-agentic-ai

[8] Cambridge Intelligence (2025). Building End-to-End Supply Chain Geospatial Network Analysis. Retrieved from https://cambridge-intelligence.com/how-i-built-an-end-to-end-supply-chain-geospatial-network-analysis-demo-in-mapweave/

[9] Chiobi, N.F. (2016). Integrating Geospatial Analytics and Business Intelligence for Workflow Optimization in Pharmaceutical Supply Chains. Scholars Journal of Economics, Business and Management.

[10] Here Austin TX (2026). Enhancing Pharmaceutical Supply Chains with Geospatial Analytics. Retrieved from https://www.hereaustintx.com/geospatial-analytics-pharmaceutical-supply-chains/

[11] Posit.co. Open-Source Data Science for Pharmaceutical Companies. Retrieved from https://posit.co/use-cases/pharma/

[12] Arahi AI. Pharmaceuticals Document Review Automation. Retrieved from https://arahi.ai/ai-agent/pharma/document-review

[13] GitHub - AI Task Manager. Retrieved from https://github.com/itallonardi/AI-task-manager